

llm-serving-ops-study 知识卡片 · 终稿

用途:LLM 推理层部署运维实战的概念复习卡 卡片总数:82 张(15 篇 lessons) 生成日期:2026-08-30 碎片指针: `scratch/0001-00015.cards.md`

CUDA(Compute Unified Device Architecture)

是什么: CUDA 是英伟达 GPU 计算平台,让显卡从画图工具变成通用数学计算器。

解决什么问题: LLM 推理全是矩阵运算,只跑在支持 CUDA 的英伟达显卡上,AMD 与苹果芯片不掌握这套语言。

我们的办法: 租用云端支持 CUDA 的 GPU 机器作为学习机,装推理框架时匹配驱动版本即可。

钥匙印模[自造]登录(SSH Public Key Authentication)

是什么: 钥匙印模[自造]登录是基于非对称密钥的远端登录方式,把自己机器的公钥存进服务器的锁匠铺[自造]后免输密码。

解决什么问题: 每次连 GPU 学习机都要敲密码既麻烦又易被截,免密方案让开门靠钥匙比对而非口令。

我们的办法: 把本地生成的公钥送到云端机器,以后凭钥匙开锁直达。

车位自造

是什么: `nvidia-smi` 输出的三格面板是显卡仪表盘,服务出问题先看这三列。

为什么重要: 推理服务出问题九成线索在显卡面板,看不到仪表盘两眼一抹黑,无法判断服务挂在哪。

我们怎么用: - 显存余量:像停车场车位,默认 0 辆空、总车位 24G;模型权重先占一坨,剩的留给 KV cache。 - GPU-Util:像工厂开工率,0% 闲、100% 连轴转,该加机器。 - Processes:像车位占用表,服务起不来时常因僵尸进程[自造]赖着占车位。

僵尸进程自造

是什么: 僵尸进程[自造]是上一次推理服务崩溃后没退干净的残留进程,仍占着显存车位。

怎么出现: 服务异常被杀或手动 kill 失败时,GPU 上的 Python 子进程漏网,显存被吃掉但服务面板显示空闲。

我们的对策: 服务起不来时先在 `nvidia-smi` 的 Processes 列找到残留 PID,用 `kill -9` 强制清场再重启。

报成功自造

是什么: 报成功[自造]≠真成功[自造]指任何安装或启动命令,屏幕打印成功信息不等于真的跑通。

为什么重要: 看不到亲眼的验证,后面出问题时只能瞎猜;验证习惯是运维的第一道防线。

我们怎么用: - 安装后敲版本号确认命令能跑,如 `docker --version`。 - 配置后用真实命令连通,如 `ssh gpu` 能进入学习机才算免密成功。 - 服务起来后拉一次真实请求验证,而不是只看启动日志。

Docker 容器(Docker Container)

是什么: Docker 容器用 namespace 划房间[自造]做进程隔离,让进程在房间内看不到房外。

解决什么问题: 多服务共用一台机器时需要让进程互不见彼此资源,它提供这种边界。

我们的办法: 本机放弃 Docker 方案改为直装 vLLM 推理框架,Docker 概念仅用于理解隔离机制。

Linux Trio(Capabilities, Seccomp, Namespace)

是什么: Linux 进程权限是指 capabilities、seccomp、namespace 三种内核机制共同限定的边界。

解决什么问题: 没有这三种机制,任何进程都能直接动内核,系统毫无安全可言。

我们的办法: 用 `capsh` 看权限位与 `/proc/self/status` 的 `Seccomp` 字段快速摸底进程权限。

Pre-Flight Trio(Diagnosis Trio)

是什么: Docker 装不上诊断三连是租到新机器时必跑的四条命令,用来一次性判断机器能否跑容器。

为什么重要: Docker 装不上常见原因有三种典型场景,三条命令可一一对应覆盖: 嵌套在容器 / 内核禁 `unshare` / `seccomp` 严格。

我们怎么用: - `cat /proc/1/comm` 输出 `tini` = 你本身已在容器里。 - `unshare -m true` 失败 = 这机器跑不了 Docker。 - `grep Seccomp /proc/self/status` 看到 `2` = 严格模式。

看日志最后一行自造

是什么: 看日志最后一行[自造]是排障时只看系统日志尾部一条的实战习惯。

为什么重要: 长日志里前面都是过程噪音,真正的报错永远在最后一行,看错行等于白看。

我们怎么用: `docker` 启动报 `done` 但连不上时,直接 `tail` 日志最后一行,通常就是死因所在。

缺权限不硬刚自造

是什么: 缺权限不硬刚[自造]指遇到权限边界时立刻评估能不能绕、不能绕就换路线的策略。

为什么重要: 内核层封死的边界硬刚永远不成功,死磕只浪费时间,识别边界类型是排障前置条件。

我们怎么用: `iptables` 配置层缺权限就改 `daemon.json` 绕过,`seccomp` 内核层封锁就放弃 Docker 改直装。

套娃被禁自造

是什么: 套娃被禁[自造]指 Docker 想在已受限容器内启动 `dockerd` 时,被内核 `unshare` 系统调用直接拒绝的现象。

怎么出现: 便宜租卡平台本身就跑在容器里,内核默认禁嵌套 `unshare`;表现为 `mount` 与 `unshare` 都报 `operation not permitted`。

我们的对策: 在这种环境直接放弃 Docker 方案,改为直装推理框架跑推理服务。

registry-mirrors(Docker Hub Mirror)

是什么: `registry-mirrors` 是写在 Docker `daemon.json` 里的镜像加速器地址,让 Docker Hub 拉取走国内中转。

解决什么问题: Docker Hub 源站在境外直连超时严重,国内中转让镜像下载又快又稳。

我们的办法: 写进 `daemon.json` 配置好后新机器直接生效。

vLLM 推理引擎(vLLM)

是什么: vLLM 是 UC Berkeley 开源的 LLM 推理引擎,核心靠物业划车位[自造]管显存、班车发车[自造]排请求。它的作用是把模型权重变成可调用的 HTTP 服务。

解决什么问题: 模型权重只是硬盘上的死数字,无人调用等于没用;vLLM 用两项技术让多用户共享服务不打架。

我们的办法: 本机直装 vLLM,一条命令即可起最小形态的推理服务。

uv(uv)

是什么: `uv` 是 Astral 出品的 Python 包管理器,装包比 `pip` 快几十倍并自带虚拟环境管理。

解决什么问题: `pip` 装包慢、`venv` 切换易忘环境出错,`uv` 用一套命令同时解决。

我们的办法: 用 `uv` 建独立环境装推理框架,不切换环境直接调用命令。

HuggingFace 模型下载路径(HF Download Path)

是什么: HuggingFace 模型下载是把权重从云端拉到本地的过程,由 HF 客户端按模型名读取。镜像与缓存机制让首次拉取慢、后续秒启。

解决什么问题: vLLM 启动时按模型名找权重,缺文件则整个推理链路断掉。

我们的办法: 用 HF_ENDPOINT 走国内镜像,权重落到本地默认缓存目录。

HF xet 401(HF xet 401)

是什么: HF xet 协议 401 故障指 HuggingFace 新版 xet 下载走自家服务器、镜像代理不了导致的鉴权失败。

怎么出现: 拉模型时未关 xet 即走镜像,镜像只代理传统下载通道,触发 401。

我们的对策: 设 HF_HUB_DISABLE_XET=1 关 xet,退回传统下载通道,镜像生效不再 401。

国内镜像条件反射自造

是什么: 国内镜像条件反射[自造]是给 Python 包、HF 模型、Docker 镜像三种来源各备一条国内通道的运维习惯。

为什么重要: 三种境外源直连都不稳,镜像让部署不靠运气,新机器拉任何依赖都能快速命中。

我们怎么用: – Python 包: 装包时显式指定 PyPI 国内镜像源。 – HF 模型: 设 HF_ENDPOINT 切镜像并关 xet。 – Docker 镜像: registry-mirrors 写进 daemon.json。

管道吞进度条自造

是什么: 管道吞进度条[自造]是指用 tee 等管道把服务日志落盘时,下载进度条被吞掉看不到的现象。

为什么重要: 大模型权重 GB 级别,看不到进度易误判进程卡死,实际正在干活。

我们怎么用: 卡住时切到第二个终端用 du 看缓存目录体积,数字在涨等于活着。

服务窗口与操作窗口分离自造

是什么: 服务窗口与操作窗口分离[自造]是指前台跑服务占一个终端、另开终端发请求的双窗口协作姿势。

为什么重要: 推理服务占住终端看日志,没第二个窗口就发不出请求做端到端验证。

我们怎么用: 一个窗口起服务持续看日志。另一个窗口装客户端发请求做验证。

KV Cache(Key-Value Cache)

是什么: KV Cache 是 Transformer 推理时显存里暂存的历史 Key/Value 向量,等于把前文的读书笔记[自造]提前写好供每个新 token 回头查。

解决什么问题: 写每个新 token 都要回头读前文所有位置,无缓存等于每写一字把整段对话重算一遍,慢到无法商用。

我们的办法: 不手管缓存,交给 vLLM 的 PagedAttention 自动调度。

PagedAttention(PagedAttention)

是什么: PagedAttention 是 vLLM 的核心显存调度技术,把 KV cache 切成固定大小 block,像操作系统虚拟内存分页管理。

解决什么问题: 直接为每个请求预分配连续显存极易碎片化,大请求占位小请求凑不上,显存白白浪费。

我们的办法: 不感知底层,只看到 vLLM 自动处理 PagedAttention。

CUDA Graph(CUDA Graph)

是什么: CUDA Graph 是英伟达提供的计算"宏录制"机制,把重复的计算步骤录下来一键重放,省去 CPU 反复下指令的开销。

解决什么问题: 推理服务每步都要 CPU 调度 GPU,小请求时 CPU 下指令的延迟盖过 GPU 计算本身,吞吐上不去。

我们的办法: 不感知配置,日志里看到 CUDA Graph 占用那 0.48 GB 就知道它在工作。

显存预算公式(VRAM Budget Formula)

是什么: 显存预算公式[自造]是把一张显卡的总显存拆成权重、KV cache、杂项三段,得出 KV cache 实际可用容量的算式。

为什么重要: 任何 LLM 服务上线前都得算这笔账,模型能不能跑、并发能扛多少,完全看公式两侧的差额。

我们怎么用: – 权重段:参数量与精度决定,1.5B 模型约 3 GB。 – 杂项段: CUDA Graph、激活值与系统预留,通常 1–2 GB。 – KV cache 段:总显存减前两段,约 18 GB 量级。

Maximum concurrency自造

是什么: Maximum concurrency[自造]是 vLLM 启动日志给出的并发承载力数字,等于 KV cache 总容量除以 max-model-len。

为什么重要: 它直接告诉你这台机器最多同时能接多少个满长请求,生产上线时设上限、防雪崩的核心依据。

我们怎么用: – 满长请求:长 32768 token 时,大约 20 个请求同时跑。 – 短请求实测:平均 2000 token 时,容量能撑到三百多个。 – 估算公式:总容量除以平均请求长度,即实际并发承载力。

圈地再营业自造

是什么: 圈地再营业[自造]指 vLLM 启动时按 gpu_memory_utilization 一次性把显存圈走 92%,运行中不再向系统追加申请。

怎么出现: 用日志里的 Using max model len、Model loading took、Available KV cache memory 三行拼出总预算,验证 92% 圈地生效。

我们的对策: 不在 vLLM 跑起来后再动态分显存,启动那一刻就把车位数钉死,运行中只靠排队与换出。

OpenAI API 概念(OpenAI-compatible API)

是什么: OpenAI API 是推理引擎按 OpenAI 规范暴露的 HTTP 端点协议。两个核心端点是花名册 `/v1/models` 与对话 `/v1/chat/completions`,已成行业事实标准。

解决什么问题: 各家引擎私有接口让客户端适配成本飙升,行业统一协议让一套调用通吃所有推理服务。

我们的办法: 客户端把 base_url 指到本机推理服务,model 字段按花名册填即可直连。

curl 工具(curl)

是什么: curl 是命令行发 HTTP 请求的工具,常用 -s 安静、-X POST 选方法、-H 加请求头、-d 传请求体组合完成调用。

解决什么问题: 调试阶段需要不写代码就模拟客户端行为,curl 让你秒级确认接口真在干活。

我们的办法: 拼一段 curl 直连本机 8000 端口,管道接 json.tool 排版响应。

usage 计费单自造

是什么: usage 计费单[自造]是接口响应里的成本字段,含 prompt_tokens 与 completion_tokens 两个数值,标着本次输入与输出体量。

为什么重要: 它让单次请求成本落到 token 粒度,运维排障与商业计费都靠它对账。

我们怎么用: – 客户端预估总 token 数控制上下文长度。 – 排障看 prompt_tokens 突增定位上下文泄漏。 – 计费按 prompt_tokens 加 completion_tokens 单价结算。

finish_reason 终止机制(finish_reason)

是什么: finish_reason 是回答结束标志机制,stop 表示自然说完、length 表示撞到 max_tokens 被截断。

为什么重要: 它让回答不完整问题第一时间定位到配置项,不必怀疑模型质量。

我们怎么用: – 看到 stop 即放心入库。 – 看到 length 先检查 max_tokens 是否给小。 – 反复 length 则怀疑 prompt 引导把回答推太长。

可替换性自造

是什么: 可替换性[自造]是 OpenAI 兼容协议带来的部署层特性,换底层引擎或模型只改地址而不动调用代码。

为什么重要: 它让学会一次部署即可对接所有 OpenAI 生态客户端,部署价值被无限放大。

我们怎么用: – 服务端换引擎:客户端零改动。 – 客户端换模型:只动 model 字段。 – 上生产换地址:只改 base_url。

nvidia-smi 听诊器自造

是什么: nvidia-smi 听诊器[自造]是把每秒刷新的显卡面板当随身仪表的用法,曲线像心跳一样连续。

为什么重要: 它让任何调参或压测都配得到带时间戳的硬件观察,事后能与请求日志逐秒对账。

我们怎么用: 任何调参或压测前先开一台终端持续跑 `nvidia-smi -l 1`,请求日志与仪表曲线同框记录。

显存利用率分诊自造

是什么: 显存利用率分诊[自造]是把两数各管一件事的判读心法,显存判起没起、利用率判忙不忙。

为什么重要: 它让显存平直线不再被误读成"服务没干活",也避免把利用率抖动错判成异常。

我们怎么用: – 显存 ≈ 0 :服务没起或权重未加载。 – 利用率波动 30~90%:正常干活。 – 显存高、利用率持续 100%:算力打满,请求在排队。

显存利用率四象限自造

是什么: 显存利用率四象限[自造]是把显存与利用率两数组合的分诊矩阵,每格对应一种服务状态。

为什么重要: 它让"该不该扩容"的判断不再靠单点直觉,而是两数同框比对。

我们怎么用: – 显存 ≈ 0 、Util 0%:服务没起。 – 高显存、Util 0%:待机;高显存、Util 30~90%:正常干活。 – 高显存、Util 持续 100%:算力打满,请求排队,该扩容。

功耗档位说话自造

是什么: 功耗档位说话[自造]是把显卡的 P-state 与瓦数当作显卡语言的判读法,P0 满载、P2 行驶、P8 怠速。

解决什么问题: 利用率易被瞬间脉冲误导,功耗作为稳态信号能区分真忙与瞬抖。

我们的办法: 盯住 P-state 与瓦数,P8 怠速低瓦、P0 逼近上限,一眼判出是否真干活。

餐厅摆桌[自造]类比(Restaurant Table Analogy)

是什么: 餐厅摆桌[自造]是把显存比作固定桌数、利用率比作上座率的类比,桌子数量不变变的是翻台节奏。

解决什么问题: 解释为什么推理时显存占用平直线是健康状态,新来者常误以为服务没动。

我们的办法: 看见显存横盘、利用率跳动,视作餐厅正常翻台,不要怀疑服务掉了。

prefill 与 decode 两段旅程(Pre-fill vs Decode)

是什么: prefill 与 decode 是大模型一次推理切出的两个分工,前段一次性审完所有输入,后段逐字生成回答。

为什么重要: 它把看似一气呵成的请求变成可分别观测与调参的两段,定位瓶颈才能对症下药。

我们怎么用: – 一次真实请求走两段:前者并行算所有输入 token。 – 后者循环十六次,逐字吐出十六 token 回答。
– 总耗时大致等于 TTFT 加上十六倍 TPOT。

TTFT(Time To First Token)

是什么: TTFT 是从发出请求到首个输出 token 出现的等待时间,反映 prefill 阶段耗时。

解决什么问题: 让"第一个字半天出不来"这种卡法有可量化的指标,排查时不再凭感觉判断。

我们的办法: 把一次真实小请求从发问到首字出现的间隔,直接当作 prefill 耗时的实测值。

TPOT(Time Per Output Token)

是什么: TPOT 是相邻两个输出 token 之间的时间间隔,反映 decode 阶段的吐字速度。

解决什么问题: 让"字吐得一顿一顿"这种卡法有可量化的指标,排查时不再凭感觉判断。

我们的办法: 把一次真实请求里相邻两字之间的间隔,直接当作 decode 速度的实测值。

流式输出(SSE)

是什么: SSE 是边算边把结果推给客户端的机制,等于服务端算出第一个字就立刻吐给用户。

解决什么问题: 用户不必等模型把整句算完才能看到结果,首字延迟可以被肉眼直接消化。

我们的办法: 在请求里加 stream: true,亲眼看到等一小段然后逐字出现的节奏。

prefill/decode 资源差异(Compute vs Bandwidth Bound)

是什么: prefill/decode 是大模型推理的两段分工。前者吃算力狂算矩阵,后者吃显存带宽反复读 KV cache。

为什么重要: 看懂两段各自卡在算力与带宽,调参方向完全不同。

我们怎么用: – prefill 慢:输入 token 太多或机器算力打满,扩容靠算力。 – decode 慢:显存带宽被多请求挤占,扩容靠带宽或换模型。 – 监控分别盯 TTFT 与 TPOT,各自反映两段瓶颈。

长 prompt 贵得有道理自造

是什么: 长 prompt 贵得有道理是说输入越长 prefill 越贵的事实,等于一万 token 系统提示词每次请求先审一万字。

为什么重要: 它让 Agent 应用里堆系统提示词的成本看得见,prompt 越长每请求 TTFT 越大。

我们怎么用: – 一万 token 系统提示词:每次请求 prefill 都要先审一万字。 – 短问题长系统词:TTFT 主要被提示词长度拖慢。 – 设计上提示词过长的服务:扩容靠算力或精简 prompt。

卡要分两种(Two Flavors of Slowness)

是什么: "卡"指用户感知的两种卡顿,首字慢与吐字慢对应不同阶段与不同资源瓶颈。

怎么出现: 首字慢对应 prefill 阶段问题,通常是输入太长或机器算力打满;吐字慢对应 decode 阶段问题,通常是显存带宽被多请求挤占。

我们的对策: 两类卡分开看,首字慢查长度或算力,吐字慢查带宽与并发。

Prometheus 暴露格式(Prometheus exposition format)

是什么: Prometheus exposition format 是监控界通用的文本协议,任何服务按此格式暴露 `/metrics` 端点即可被通用工具读取。

解决什么问题: 不同服务的指标格式各自为政,无法被统一工具读取。统一格式让任何系统能被同一种工具消费。

我们的办法: vLLM 服务原生把账本[自造]摊开在 `/metrics` 端点,直接 curl 就能读到这套文本协议。

指标三类型分诊自造

是什么: 指标三类型分诊[自造]是把服务指标按更新方式归类的第一刀,服务一上来就该先问这三类问题。

为什么重要: 三类问法对应不同场景,分错类型就会问错问题。

我们怎么用: - `vllm:kv_cache_usage_perc` 当 gauge 看——盯当前缓存占几成 - `vllm:prompt_tokens_total` 当 counter 看——除以时间得吞吐 - `vllm:time_to_first_token_seconds` 当 histogram 看——看分布而非平均

监控原材料自造

是什么: 监控原材料[自造]是说服务 `/metrics` 输出的指标本质就是后续工具要加工的素材,先读懂原料再让工具接手。

为什么重要: 不读原料直接装工具,等于不会看体温就买血压计。后续曲线失去解读根,出问题也只能裸 curl 救急。

我们怎么用: 在裸机或工具失效时,一条 curl 就拿到服务状态,不被监控系统绑架。

指标四件套自造

是什么: 指标四件套是指在 vLLM `/metrics` 输出里最先该认准的四类指标,作为入门体检清单。为什么重要: 几百行盲翻效率为零,先认准这四个就能覆盖服务健康的关键判断。

我们怎么用: - `vllm:kv_cache_usage_perc` —— KV 缓存占用比,逼近 1 表示要排队 - `vllm:num_requests_running / waiting` —— 在跑 / 排队的请求数 - `vllm:prompt_tokens_total / generation_tokens_total` —— 累计输入 / 输出 token,算吞吐 - `vllm:time_to_first_token_seconds` —— TTFT 分布,首字延迟体检

平均值掩盖尾巴自造

是什么: 平均值掩盖尾巴[自造]是说汇报延迟只看平均数时的认知陷阱,长尾请求的糟糕体验被多数的快请求平均掉。

怎么出现: 故障路径——读 TTFT 只看平均值,histogram 里 99 分位的几秒慢请求被完全吞掉;实测 1% 请求卡 5 秒时,平均仍能装作 0.3 秒。

我们的对策: 在指标阅读层防,办法是读 histogram 分桶而非均值,防不住的是只看均值的旧习惯。

Prometheus 监控系统(Prometheus)

是什么: Prometheus 是 CNCF 毕业的项目,身兼抄表员[自造]与仓库两种角色,定时主动拉取各服务指标并按时间存为本地 TSDB。

解决什么问题: 手动 curl 只能看瞬时状态,事故复盘需要"出事前 10 分钟发生了什么"的历史记录,Prometheus 把指标变时间序列留底。

我们的办法: 本项目起一个 Prometheus 实例,定时主动去 vLLM 的 `/metrics` 抄账,数据默认在本地 TSDB 留 15 天。

PromQL 查询语言(PromQL)

是什么: PromQL 是 Prometheus 自带的查询语句,专门对时间序列数据库提问用。

解决什么问题: 抓到的指标是裸数据,要切片、聚合、算增速时,缺一种语言把这些事统一表达出来。

我们的办法: 本项目先用最简形态直接写指标名做即时查询,后面会用到 `rate(x[5m])` 算 5 分钟增速。

pull 模型 vs push 模型(Pull vs Push)

是什么: pull 模型是监控方当抄表员[自造]主动上门抄表的架构,push 模型是服务方自己报案[自造]的架构。

为什么重要: 抄表员主动上门,谁家没开门(服务挂了)立刻知道;出事方往往自己报案也来不及,pull 让"挂掉"自带告警,简单可靠。

我们怎么用: 用 Prometheus 默认的 pull 形态,定时主动拉 vLLM 的 `/metrics`;抄不到就等于服务出事。

配置即登记自造

是什么: 配置即登记[自造]是说 Prometheus 的 yaml 配置本质是一张抄表清单[自造],改监控等于改清单条目。

为什么重要: 它让"加监控"从神秘运维动作变成透明文件编辑,新加谁抄谁直接看 yaml 就懂,排查与交接都顺。

我们怎么用: - vllm 这个 job 名代表一类抄表任务。 - localhost:8000 这个 target 指明抄谁。 - scrape_interval 15s 控制抄表频率。

Grafana 可视化平台(Grafana)

是什么: Grafana 是开源可视化工具,接数据源(尤其 Prometheus)把数字画成仪表盘。

解决什么问题: 裸数字看趋势费劲,曲线一眼能见涨跌与异常,运维要的是驾驶舱而非账本。

我们的办法: 本项目用 Grafana 接 Prometheus,把 PromQL 查询结果画成第一个面板。

SSH 端口转发(SSH Local Port Forwarding)

是什么: 这是借 SSH 加密隧道把远端端口借到本机的招数,英文叫 local port forwarding。

解决什么问题: 远端服务在防火墙后,网页端口不对外开放,本机浏览器借 SSH 这条隧道就能访问。

我们的办法: 用 `ssh -L 3000:localhost:3000 gpu -N` 把远端 Grafana 借到 Mac,浏览器开 localhost:3000 即可。

Grafana 不存数据[自造]

是什么: Grafana 不存数据[自造]是说 Grafana 只负责画,持久化由数据源仓库承担。

为什么重要: 它让画图与存数职责分明,任一端挂掉数据都不丢,故障爆炸半径小。

我们怎么用: 用 Grafana 只当 Prometheus 的画图前端;Prometheus 出问题 Grafana 也不丢历史曲线。

指标名漂移(Metric Name Drift)

是什么: 这是框架升级时指标被改名的现象,旧 PromQL 查询瞬间 no data。

怎么出现: vLLM 0.27 把 `gpu_cache_usage_perc` 改名为 `kv_cache_usage_perc`,旧教程的查询全 no data。

我们的对策: 任何 PromQL 查询先 grep 服务自己的 `/metrics` 拿真名,再用真名建面板。

rate 速率计算(rate)

是什么: rate 是 PromQL 的速率函数,把 counter 里程表算成平均每秒的增量。

解决什么问题: counter 类型只增不减,直接画是爬坡线看不出"现在多快",rate 把累计数转成实时增速。

我们的办法: 对 token 累计类 counter 都套 rate,曲线才看得出每秒吞吐。

histogram_quantile 分位数计算(histogram_quantile)

是什么: histogram_quantile 是 PromQL 的分位数函数,从 histogram 分桶反推 P99 等延迟门槛值。

解决什么问题: histogram 只告诉你"多少请求 <0.1s、<0.5s",histogram_quantile 算得"P99 是几秒"这条具体线。

我们的办法: TTFT 与 TPOT 延迟都套 histogram_quantile(0.99, sum by (le) (rate(...[5m]))) 看最慢 1%。

监控四件套自造

是什么: 监控四件套[自造]是说 Grafana 面板上必看的四条曲线,前两条管堵不堵、后两条管快不快。为什么重要: 四条线同时抬头才算亲眼验证了压测到反应的因果链,空跑的监控永远只是装饰画。

我们怎么用: – KV 缓存使用率 — 看车位占用,抬到顶就是排队的信号 – TTFT(P99) — 看审题延迟,峰值代表最慢用户的等待 – TPOT(P99) — 看吐字速度,峰值代表生成卡顿 – 吞吐 tokens/s — 看整体算力,峰值代表压测被服务吸收

盯 P99 不盯平均自造

是什么: 盯 P99 不盯平均[自造]是说监控延迟必须看分位数而不是均值,均值被快请求洗白会撒谎。为什么重要: P99 才是用户真实体验的底线,平均漂亮不代表没人倒霉,histogram 套 quantile 是入门必备。

我们怎么用: 所有延迟曲线都从 histogram 取 P99,平均数只在解释总体趋势时作为辅助参考。

counter 套 rate、histogram 套 quantile自造

是什么: counter 套 rate、histogram 套 quantile[自造]是 PromQL 入门两个固定搭配,记住就不会瞎查账。为什么重要: counter 不套 rate 永远是爬坡线,histogram 不套 quantile 拿不到具体延迟值,类型与函数错配就问错问题。

我们怎么用: – counter 类指标(如 token 累计)统一套 rate(...) 求每秒增量 – histogram 类指标(如延迟分桶)统一套 histogram_quantile(...) 求分位数 – 搭配固定,见到类型就反射出对应函数

服务的呼吸自造

是什么: 服务的呼吸[自造]是说并发砸来时指标先涨、并发退去时指标回落,这一上一下就是服务的健康心跳。为什么重要: 看见曲线回落才知道服务真的能恢复,光涨不落代表已经卡死。

我们怎么用: – 压测开始 → 四条曲线一起抬头 – 请求完成 → 曲线陆续回落 – 回落速度低于新请求速度 → 排队累积、必须扩容

--max-model-len 上限(--max-model-len)

是什么: --max-model-len 是 vLLM 给单请求长度设的天花板,单位 token,默认取模型自身最大值。

解决什么问题: 模型默认能吃很长上下文,但业务用不上的长度会让并发承载力被白白压低。

我们的办法: 在 Qwen2.5-1.5B 上把 32768 砍到 4096,让启动日志里的最大并发从 20 涨到 166。

并发承载力公式自造

是什么: 并发承载力公式[自造]是把显卡 KV 缓存总容量除以单请求上限,得出这台机器同时能跑多少个满长请求的算式。

为什么重要: 它把"显存已圈定"和"单请求长度"两件事合成一个直观比值,改任何一边都能立刻看出天花板的位移。

我们怎么用: – 砍预算:从 32768 砍到 4096,承载力从 20 涨到 166。 – 加显存:同样能放大并发,但要花钱换卡。 – 不花钱的优化:永远先砍预算,再考虑加显存。

大货车 vs 家用轿车自造

是什么: 大货车 vs 家用轿车[自造]是把 KV 缓存容量比作停车场、按单请求长度类比车长的比喻。

解决什么问题: 总车位固定时按"40 米长大货车"[自造]只能停 20 辆,按"家用轿车"[自造]能停 166 辆,直观说明改假设比改硬件划算。

我们的办法: 用这个比喻理解为什么调参就能拉开几倍的并发差距,不需要换显卡。

调参 = 改假设自造

是什么: 调参 = 改假设[自造]是说这类参数不改模型、不改代码,只改对流量预期的认知。

为什么重要: 它把这件事从碰引擎参数拉回到理解业务,业务上限天花板比 GPU 架构更直接。

我们怎么用: – 上限按业务定:Agent 对话多在千级 token,把 32K 砍到 4K 足矣。 – 设大了浪费:并发按最长的算,白白给承载力打折。 – 改配置前问业务:最长对话多少 token,留点余量就设多少。

超长请求 400 报错(400 on Oversized Request)

是什么: 这是配置层主动拦截,长度超过 max-model-len 设的天花板后,vLLM 直接拒绝处理、返回 HTTP 400。

怎么出现: 把 max-model-len 从默认砍小后,任何 prompt 加输出超出上限的请求都会被立刻挡在门外,这是预期行为不是服务故障。

我们的对策: 不把这种 400 当成事故排查,直接告诉调用方缩短请求或调高上限。

--gpu-memory-utilization 显存水位(--gpu-memory-utilization)

是什么: --gpu-memory-utilization 是 vLLM 启动参数,0~1 的比例,告诉 vLLM 圈地时最多占总显存的几成。

解决什么问题: vLLM 默认 0.9/0.92 奔着独占整卡去,共卡部署时其他进程拿不到显存会直接 OOM。

我们的办法: 共享显存时把这个值主动调低,把显存让给 embedding 等其他进程。

固定成本与可变成本自造

是什么: 固定成本与可变成本[自造]是说把显存按能否压缩拆两块算账的比喻,权重与杂项是雷打不动的固定成本。

为什么重要: 砍预算时只能砍可变成本,这把账算清楚就知道改哪个参数才真的有用。

我们怎么用: – 权重是固定成本:模型文件常驻,不能动。 – 杂项是固定成本:激活、CUDA Graph 等,不能动。 – KV 缓存是可变成本:并发进来就涨、出去就降,唯一能动。

一次只动一个变量自造

是什么: 一次只动一个变量[自造]是说做对比实验时只改一个参数,其他保持不变,才能把功劳归对。

为什么重要: 同时改两个变量,结果变了你分不清是谁的功劳,实验就白做了。

我们怎么用: 本次保留上下文长度参数不动,只调显存水位这一项,确保观察到的变化只能来自水位。

共卡部署成本核算自造

是什么: 共卡部署成本核算[自造]是说一张卡同时跑两个模型,各自水位设多少、并发损失多少全靠显存预算公式算。

为什么重要: 共卡不靠公式就是凭感觉,要么把显存让过头、要么让不到位挤爆副进程。

我们怎么用: – 主模型 92% 水位、副模型留 5~10%,按预算公式算清双方并发上限。 – 两个模型大小相近时,各占 50% 容量,各承受并发腰斩。 – 公式一旦定下,改配置前先算账再动手,避免事后回滚。

逼近显存极限的 OOM 风险自造

是什么: 逼近显存极限的 OOM 风险[自造]是说把 --gpu-memory-utilization 调到接近 1 时,看似榨干显存、实则可能随时 OOM 的陷阱。

怎么出现: 调高到 0.95 以上时,留给权重和杂项的余量被压缩,CUDA 上下文波动就立刻 OOM。

我们的对策: 共卡场景下主动调低留出余量,独占部署也最多停在 0.92,绝不逼近 1。

--max-num-seqs(最大并发序列数)

是什么:vLLM 启动参数,指调度器一次最多允许同时计算的请求数。超出的请求进入等待队列,而不是被直接拒绝。

解决什么问题:防止请求无上限涌入挤爆显存与 KV cache,保住在算请求的速度。

我们的办法:启动时显式把上限调到 8,做 30 并发压测看排队行为。

continuous batching(连续批处理)

是什么:vLLM 的核心调度机制,指每算完一个请求立刻从队列补位下一个。没有空等浪费,像电梯有人到顶立刻下、排队立刻上。

解决什么问题:避免传统批处理"等人齐再发车"的空等浪费,把吞吐拉满。

我们的办法:实验中 8 个在算、22 个排队,有人完成立刻补位就是它的体现。

排队 ≠ 故障自造

是什么:排队[自造]是限流带来的正常等待,不是服务异常的信号。

为什么重要:它把"在算请求被保护"与"请求真卡死"区分开,让告警与排障有据可依。

我们怎么用: - TTFT 包含等待时长,告警先看 waiting 曲线,确认只是等待而非卡死 - running 顶死在 max-num-seqs 是限流生效的证据 - 在算请求的速度不变,用户体验有保证

吞吐与延迟跷跷板自造

是什么:max-num-seqs 等并发参数调大追吞吐、调小保延迟,两端无法同取极值。

为什么重要:它把"调参"从凭感觉变成按业务目标选点的明确决策。

我们怎么用: - max-num-seqs 调小,延迟稳定但吞吐下降 - max-num-seqs 调大,吞吐上升但单请求变慢 - 线上选点看业务:对话类偏延迟,批量任务偏吞吐

请求被赶出去重算自造

是什么:不限流时显存与 KV cache 紧张,vLLM 把算到一半的请求踢出去回收资源。

怎么出现:不限流的 30 并发压测触发抢占,被踢请求必须从头重算。

我们的对策:用 max-num-seqs 限制并发上限,让超额请求排队而非被抢占。

前缀缓存(Automatic Prefix Caching, APC)

是什么:APC 是 vLLM 内置功能,指逐字相同的 prompt 开头的 KV 计算结果可被后续请求复用。

解决什么问题:Agent 多请求共享长 system prompt 时,避免每次都重复 prefill 同一段内容。

我们的办法:不传参默认开启,跑对照实验对比 TTFT 与 prefix_cache_hits_total。

前缀缓存的应用延伸(Prefix Cache Application Layer)

是什么:它是 PagedAttention 的应用延伸,指 vLLM 按 prompt 前缀做块键查询,命中即复用 KV。

解决什么问题:让多次请求的相同开头不必重复 prefill,直接命中已算好的 KV 块。

我们的办法:看 prefix_cache_hits_total 上涨直接验证块级复用命中。

开头相同才生效自造

是什么:前缀缓存复用的硬条件,指 prompt 必须从第一个 token 起逐字一致,中间相同不算。

为什么重要:它把"哪些请求能共享缓存"从经验判断变成字节级契约,让 prompt 设计可验证。

我们怎么用: – 时间戳放前面 → 缓存失效 – 随机 ID 放开头 → 永远命中 0 – 中段相同 → 仍不命中

不变内容放最前自造

是什么:这是调用方唯一需要遵守的契约,指把不变的 system prompt 与工具定义放在 prompt 最前面。

为什么重要:它把"用上缓存"从框架黑盒变成调用方可控的工程动作,直接转化为 TTFT 与成本。

我们怎么用: – system prompt 含角色与工具说明数千 token,放最前 – 用户问题放最后,变化越大越靠后

前缀复用 A/B 实验自造

是什么:用两组对照验证复用收益的实验打法,指用各异长 prompt 与共享长开头 prompt 对比 TTFT。

为什么重要:它让"按前缀能省多少"从口算变成可量化的指标差,决策有据可依。

我们怎么用: – A 组 30 个各异长 prompt → TTFT P99 作基线 – B 组 30 个共享长 system prompt → 看 prefix_cache_hits_total 上涨 – 两组对照 → 量化收益证明复用生效

Agent 场景省钱王自造

是什么:Agent 类应用的省钱特性,指固定 system prompt 越长,前缀缓存省下的 prefill 计算越多。

为什么重要:它把 prompt 结构设计从纯语义问题升级成应用层性能优化手段,直接挂钩成本。

我们怎么用: – system prompt 越长,单请求节省越可观 – 与不变内容放最前配套使用 – prompt 工程师写法习惯直接转化为云端账单